SecureShare: A DevSecOps Approach to Building an Attack-Resistant File-Sharing Service

Midterm Lab Exam — CYC386 Secure Software Design and Development (Fall 2026)
COMSATS University Islamabad — Department of Computer Science

Team Codex

Author	Roll No
Zain Ali	FA23-BCT-059
Nimra Umar	FA23-BCT-031

Instructor: Engr. Muhammad Ahmad Nawaz Repository: `SecureApp-Sprint-Codex` *· Sprint: 7-day DevSecOps security sprint*

Abstract

This report documents a one-week DevSecOps security-engineering sprint in which Team Codex designed, implemented, tested, and hardened **SecureShare**, a file-sharing and collaboration web service. Following the Secure SDLC, we performed Protection Needs Elicitation (PNE) mapped to the CIA triad, conducted STRIDE threat modeling with Data-Flow Diagrams, attack trees, and CVSS v3.1 quantitative risk scoring, and then implemented and verified fixes for the five mandatory vulnerability classes — **Buffer Overflow (CWE-120), Command Injection (CWE-78), Insecure Direct Object Reference (CWE-639), Cross-Site Request Forgery (CWE-352), and Clickjacking (CWE-1021)** — plus additional hardening. A GitHub Actions CI/CD pipeline enforces build, test, and SAST/DAST security gates that fail on high/critical findings. The result is a deployable, attack-resistant application with full traceability from protection need → threat → fix → test. Index terms: *DevSecOps, Secure SDLC, STRIDE, CVSS, SAST, DAST, memory safety, authorization.*

Table of Contents

1. Introduction
2. System Architecture
3. Protection Needs Elicitation & CIA Mapping
4. Threat Model & Quantitative Risk
5. Secure Implementation (Detailed Remediation)
6. CI/CD Pipeline & Automation
7. Security Testing Evidence (SAST/DAST)
8. CIA Residual-Risk Analysis
9. Lessons Learned
10. Individual Contribution Statement
11. Conclusion
12. References Appendix A — Repository Map · Appendix B — Reproduction Steps

1. Introduction

1.1 Context and Motivation

Modern software is attacked continuously, and security defects discovered late are expensive and dangerous. DevSecOps shifts security *left* — integrating threat modeling, secure coding, and automated security testing into the development lifecycle rather than bolting them on afterwards. For this midterm sprint we acted as a DevSecOps team handed a starter application to harden to production-grade security within seven calendar days, culminating in a security review and demo.

1.2 Application Chosen

We built **SecureShare**, a file-sharing/collaboration service (exam theme "File Sharing / Collaboration"). This domain naturally exposes all five mandatory vulnerability classes: a native parser/preview component (buffer overflow), a file export path that invokes an external process (command injection), object access by identifier (IDOR), state-changing share/delete actions (CSRF), and sensitive pages that could be framed (clickjacking).

1.3 Objectives

- Perform PNE and map every protection need to Confidentiality, Integrity, and Availability (CDF Unit 2).
- Threat-model with STRIDE, DFDs, attack trees, and CVSS v3.1 (CDF Unit 3).
- Demonstrate (lab-only) and remediate the five mandatory classes plus additional findings (CDF Unit 5).
- Build a CI/CD pipeline with SAST + DAST that fails on critical findings.
- Maintain professional Git/GitHub workflow and a traceable Jira board.

1.4 Scope and Ethics

All exploitation was confined to our own instance on `127.0.0.1`, consistent with Section 9 of the exam. Deliberately vulnerable code is isolated under `vulnerable_demo/` and is never deployed or imported by the running application.

---

2. System Architecture

2.1 Overview

SecureShare is a Python **Flask** application using **SQLAlchemy**/**SQLite** for persistence, **Flask-Login** for session authentication, **Flask-WTF** for CSRF protection, **Flask-Limiter** for rate limiting, and **Argon2** for password hashing. A small **C** helper (`thumbgen_secure`) generates a bounded preview of uploaded files and is invoked via `ctypes`; a safe pure-Python fallback runs when the library is not compiled.

2.2 Components

Layer	Technology	Responsibility
Presentation	Jinja2 templates + CSP	Auth, dashboard, share console
Application	Flask blueprints (`auth`, `files`)	Business logic, authorization
Native	C shared library (`ctypes`)	Bounded file preview
Data	SQLAlchemy ORM + SQLite	Users, files, shares
Storage	Filesystem (`uploads/`)	File bytes (UUID names)
Pipeline	GitHub Actions	Build, test, SAST, DAST gates

2.3 Request Lifecycle (secure path)

1. Browser submits a request with a session cookie (`HttpOnly`, `SameSite=Lax`, `Secure`) and, for mutations, a CSRF token.
2. Flask-Login authenticates; `CSRFProtect` validates the token.
3. The route performs **object-level authorization** before touching an object.
4. Responses carry a strict security-header set (CSP, `X-Frame-Options`, `nosniff`, HSTS).

2.4 Trust Boundaries

TB-1 Internet↔App, TB-2 App↔Native/subprocess, TB-3 App↔Datastore. Each is the focus of the STRIDE analysis (§4). *(Context and Level-1 DFDs appear in `THREAT_MODEL.md` §2–3.)*

---

3. Protection Needs Elicitation & CIA Mapping

3.1 Method

We inventoried assets (data, processes, components, external entities), elicited protection needs from user, administrator, and adversarial perspectives, and mapped each need to the CIA triad, then derived **measurable** security requirements. The full tables are in `PNE_Report.md`; a summary follows.

3.2 Key Assets

Credentials and session tokens (Critical), uploaded file content and share grants (High), the datastore and the authentication/authorization processes (Critical), the native helper and export process (High), and the CI/CD pipeline with its secrets (High).

3.3 Protection Needs → CIA (summary)

PN	Need	C	I	A
PN-01	No recoverable password storage	✔	✔	
PN-02/03	Files only for owner/grantee; no IDOR	✔	✔	
PN-04/05	No command/memory exploitation via input	✔	✔	✔
PN-06	Genuine-origin state changes (anti-CSRF)		✔	
PN-07	No malicious framing (clickjacking)		✔	
PN-08	Session integrity/confidentiality	✔	✔	
PN-09	Availability under abuse			✔
PN-10	No secret leakage	✔		
PN-11	Safe upload handling	✔	✔	
PN-12	No security regressions to `main`	✔	✔	✔

3.4 Measurable Requirements

Eleven requirements (SR-01…SR-11) were derived, each with a verification method and a Jira issue (e.g., SR-02 "object-level authorization on every access", verified by `test_idor_other_user_is_forbidden`). See `PNE_Report.md` §5.

---

4. Threat Model & Quantitative Risk

4.1 STRIDE

STRIDE was applied to every DFD element (`THREAT_MODEL.md` §4). Highlights: Authorization process → **Information Disclosure**/**Elevation** (IDOR); Export process → **Tampering**/**Elevation** (command injection); Native helper → **Tampering**/**DoS**/**Elevation** (buffer overflow); Web UI → **Tampering** (CSRF, clickjacking, XSS).

4.2 Attack Trees

Two attack trees were constructed: (A) *RCE via Command Injection* and (B) *Control-flow hijack/ DoS via Buffer Overflow*. Each leaf is annotated with the mitigation that breaks the path (`THREAT_MODEL.md` §5).

4.3 CVSS v3.1 Scoring

Risk	CWE	Vector	Base	Severity
Command Injection	78	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H`	8.8	High
SQL Injection (regression guard)	89	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N`	8.1	High
Buffer Overflow	120	`AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:H`	7.0	High
IDOR	639	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`	6.5	Medium
CSRF	352	`AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N`	6.5	Medium
Brute-force/DoS	307	`AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:L`	5.4	Medium
Clickjacking	1021	`AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N`	4.3	Medium

4.4 Prioritized Risk Register

Risks were ranked by base score and likelihood into P1–P3 and linked to Jira issues and remediation PRs (`THREAT_MODEL.md` §7). All P1/P2 items are fixed and verified.

---

5. Secure Implementation (Detailed Remediation)

Each subsection summarizes the fix; full before/PoC/after/verify is in `SECURITY_IMPLEMENTATION.md`.

5.1 Buffer Overflow (CWE-120)

Before: `memcpy(buf, in, in_len)` into a 64-byte stack buffer trusting a caller-supplied length, plus `strcpy` ignoring the output capacity. *After:* the copy length is clamped to the buffer capacity, the result is NUL-terminated in bounds, and output is written with `snprintf(out, out_cap,…)`. The Python wrapper caps input at 256 bytes. The library is compiled with `-fstack-protector-strong -D_FORTIFY_SOURCE=2`; NX/DEP and ASLR are provided by the loader. *Evidence:* AddressSanitizer reports a stack overflow on the vulnerable build and a clean run on the secure build.

5.2 Command Injection (CWE-78)

Before: `subprocess.check_output(f"zip … {filename}", shell=True)`. *After:* the user chooses only an **allow-listed format key**; the export runs `subprocess.run([...], shell=False, timeout=15)` with an argument array, so shell metacharacters are inert. *Evidence:* Bandit flags the vulnerable code (B602/B605, High); the secure route rejects a non-allowlisted format with 400 and a direct test proves injected commands never execute.

5.3 IDOR (CWE-639)

Before: `send_file(file_by_id)` with no ownership check. *After:* `File.is_accessible_by(user)` (owner, admin, or explicit share grantee) is enforced on every read; owner-only on mutations. *Evidence:* a non-owner receives 403; the owner still receives 200.

5.4 CSRF (CWE-352)

Before: state-changing POSTs without an anti-CSRF token. *After:* global `CSRFProtect` synchronizer tokens on every form plus `SameSite=Lax` cookies. *Evidence:* a token-less POST returns 400; a genuine same-site POST succeeds.

5.5 Clickjacking (CWE-1021)

Before: sensitive pages framable. *After:* `X-Frame-Options: DENY` and CSP `frame-ancestors 'none'` on every response. *Evidence:* the browser refuses to frame the dashboard; header tests pass on public and authenticated pages.

5.6 Additional Hardening

Argon2id hashing, rate limiting, upload allow-list + size cap + `secure_filename`

- UUID storage names, parameterized ORM (anti-SQLi), full security-header set, environment-sourced secrets, and `debug=False`.

---

6. CI/CD Pipeline & Automation

6.1 Design

`.github/workflows/ci-cd.yml` runs on push and pull_request to `main`/`develop` with five jobs: **build-and-test** (deps, hardened native build, pytest, ASAN before/after evidence), **sast-bandit** (gate on `app/`), **sast-semgrep** (OWASP Top-10 + security-audit, SARIF upload), **codeql** (python + cpp matrix), and **dast-zap** (OWASP ZAP baseline).

6.2 Fail-on-Critical

Bandit (`-ll`) and Semgrep (`--error`) exit non-zero on high/critical findings in production code, failing the required check. With branch protection on `main`, a regression cannot be merged. We demonstrate the gate by contrasting a clean `app/` scan (exit 0) with the vulnerable reference scan (exit 1).

6.3 Git/GitHub Workflow

Protected `main`, `feature/*` and `fix/*` branches, pull requests with a mandatory peer review, descriptive commits referencing Jira/CWE, and retained workflow artifacts (test results, scan reports, ASAN logs).

---

7. Security Testing Evidence (SAST/DAST)

`SECURITY_TEST_REPORT.md` holds the detail; evidence files are under `docs/evidence/`. Summary: 14/14 tests pass; Bandit clean on `app/` and failing on the vulnerable reference; Semgrep/CodeQL clean on production code; OWASP ZAP baseline moves from multiple WARN/FAIL (missing XFO/CSP/CSRF/cookie flags) to PASS after remediation; ASAN detects the overflow before and is clean after.

---

8. CIA Residual-Risk Analysis

8.1 Confidentiality

Credentials are Argon2id-hashed; files are gated by object-level authorization; secrets are environment-sourced; transport is TLS at the proxy. *Residual:* SQLite is unencrypted at rest (lab); production would encrypt at rest.

8.2 Integrity

CSRF tokens, SameSite cookies, parameterized queries, and the SAST gate protect integrity of actions and data. *Residual:* supply-chain integrity would be strengthened by `pip-audit`/ Dependabot and signed commits.

8.3 Availability

Rate limiting and the 10 MB upload cap mitigate abuse. *Residual:* in-memory rate-limit storage is single-node; production would use Redis and a WAF/CDN.

8.4 Overall

No High/Critical residual risks remain open at sprint close; residual risk is assessed **Low** for the lab deployment (`THREAT_MODEL.md` §8).

---

9. Lessons Learned

- **Threat-model first.** Mapping STRIDE to DFD elements made the five classes fall out naturally and guided where to spend effort (P1 injection/overflow).
- **Never trust a length.** The buffer overflow reduced to a single trusted size field — the fix is conceptually small but decisive.
- **Eliminate the shell, don't filter it.** Allow-lists + argv arrays beat blacklisting metacharacters.
- **Centralize authorization.** One `is_accessible_by` method is easier to audit than scattered checks and directly answers IDOR.
- **Gates change behavior.** A failing pipeline on a reintroduced vuln is far more convincing than a document claiming safety.
- **Isolate the vulnerable demo.** Keeping PoCs out of the deployed path avoided turning our own learning aids into real risk.

---

10. Individual Contribution Statement

Member	Roll No	Primary contributions
Zain Ali	FA23-BCT-059	Secure backend implementation (auth, files, native helper fix, command-injection & IDOR remediation), CI/CD pipeline (Bandit/Semgrep/CodeQL/ZAP, fail-on-critical), CVSS scoring, repo setup & branch protection.
Nimra Umar	FA23-BCT-031	PNE & CIA mapping, STRIDE analysis, DFDs & attack trees, CSRF & clickjacking remediation and session hardening, DAST/ZAP job, documentation & repo hygiene, Jira board.

Both members participated in code reviews and pull-request approvals; commit and PR history in GitHub evidences individual work.

---

11. Conclusion

Within a seven-day sprint, Team Codex delivered a hardened, deployable file-sharing service that demonstrably resists the five mandatory vulnerability classes and several additional threats, backed by a CI/CD pipeline that enforces security gates and full PNE→threat→fix→test traceability. The exercise reinforced that secure software is produced by process and automation, not by afterthought — "secure coding is a mindset, not just a toolset."

12. References

1. OWASP Top 10 (2021). OWASP Foundation.
2. OWASP Cheat Sheet Series: OS Command Injection, CSRF Prevention, Clickjacking Defense, Authorization.
3. CWE-119/120/787 (Buffer Errors), CWE-78 (OS Command Injection), CWE-639 (Authorization Bypass/IDOR), CWE-352 (CSRF), CWE-1021 (UI Redress).
4. FIRST. *CVSS v3.1 Specification Document.*
5. Microsoft. *The STRIDE Threat Model.*
6. B. Schneier. *Attack Trees.* Dr. Dobb's Journal, 1999.
7. CERT C Coding Standard; *Building Secure Software*; *Designing Secure Software.*
8. GitHub Actions, CodeQL, Semgrep, Bandit, OWASP ZAP documentation.

Appendix A — Repository Map

```
app/              Flask application (secure)
native/           thumbgen_secure.c (used) + thumbgen_vuln.c (lab PoC)
vulnerable_demo/ isolated insecure PoCs (never deployed)
tests/            pytest security suite (14 tests)
docs/             PNE, threat model, implementation, test report, this report, jira/,
evidence/
.github/          ci-cd.yml, PR template
```

Appendix B — Reproduction Steps

```
python -m venv .venv && . .venv/bin/activate   # (Windows: .
\.venv\Scripts\Activate.ps1)
pip install -r requirements.txt
pytest tests -q                  # 14 passed
bandit -r app -ll                # clean (gate)
bandit -r vulnerable_demo        # High findings (detection evidence)
make -C native secure            # build hardened helper (Linux/macOS)
python run.py                    # http://127.0.0.1:5000
```

Appendix C — Glossary

Term	Meaning
PNE	Protection Needs Elicitation — identifying what must be protected and why.
CIA	Confidentiality, Integrity, Availability — the core security properties.
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, DoS, Elevation of privilege.
DFD	Data-Flow Diagram — processes, data stores, flows, external entities, trust boundaries.
CVSS	Common Vulnerability Scoring System — standardized severity scoring (v3.1).
SAST	Static Application Security Testing — analyzing source without running it.
DAST	Dynamic Application Security Testing — testing the running application.
IDOR	Insecure Direct Object Reference — accessing objects by manipulating identifiers.
CSRF	Cross-Site Request Forgery — forcing a victim's browser to make state-changing requests.
ASAN	AddressSanitizer — a memory-error detector used during testing.
SARIF	Static Analysis Results Interchange Format — standard for scanner output.

Appendix D — Full Source Listings

The listings below are the security-critical modules, provided in full so every change can be explained and justified during the viva.

D.1 Secure response-header hook — `app/__init__.py` (excerpt)

```
@app.after_request
def set_secure_headers(resp):
    resp.headers["X-Frame-Options"] = "DENY"                  # Clickjacking
    resp.headers["Content-Security-Policy"] = (
        "default-src 'self'; script-src 'self'; style-src 'self'; "
        "img-src 'self' data:; object-src 'none'; base-uri 'self'; "
        "frame-ancestors 'none'; form-action 'self'")
    resp.headers["X-Content-Type-Options"] = "nosniff"
    resp.headers["Referrer-Policy"] = "no-referrer"
    resp.headers["Permissions-Policy"] = "geolocation=(), microphone=(), camera=()"
    resp.headers["Strict-Transport-Security"] = "max-age=63072000;
includeSubDomains"
    return resp
```

D.2 Data model & object-level authorization — `app/models.py` (excerpt)

```
class File(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    stored_name = db.Column(db.String(128), unique=True, nullable=False)
    original_name = db.Column(db.String(256), nullable=False)
    owner_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False)
    shares = db.relationship("Share", backref="file", cascade="all, delete-orphan")

    def is_accessible_by(self, user):
        if user is None or not user.is_authenticated:
            return False
        if self.owner_id == user.id or user.is_admin:
            return True
        return any(s.grantee_id == user.id for s in self.shares)
```

Passwords use Argon2id:

```
from argon2 import PasswordHasher
_ph = PasswordHasher()
def set_password(self, pw):   self.password_hash = _ph.hash(pw)
def check_password(self, pw):
    try:    return _ph.verify(self.password_hash, pw)
    except VerifyMismatchError: return False
```

D.3 Secure file operations — `app/files.py` (key routes)

```
@files_bp.route("/file/<int:file_id>/download")
```

```
@login_required
def download(file_id):
    f = db.session.get(File, file_id)
    if f is None: abort(404)
    if not f.is_accessible_by(current_user):      # IDOR fix (CWE-639)
        abort(403)
    path = os.path.join(current_app.config["UPLOAD_DIR"], f.stored_name)
    if not os.path.exists(path): abort(404)
    return send_file(path, as_attachment=True, download_name=f.original_name)

_EXPORT_CMDS = {
    "zip": ["python", "-c",
            "import sys,zipfile; z=zipfile.ZipFile(sys.argv[2],'w'); "
            "z.write(sys.argv[1]); z.close()"],
}

@files_bp.route("/file/<int:file_id>/export", methods=["POST"])
@login_required
def export(file_id):
    f = _get_owned_file_or_403(file_id)
    fmt = (request.form.get("format") or "zip").lower()
    if fmt not in _EXPORT_CMDS: abort(400)        # allow-list
    src = os.path.join(current_app.config["UPLOAD_DIR"], f.stored_name)
    out = os.path.join(current_app.config["UPLOAD_DIR"], f"{f.stored_name}.{fmt}")
    cmd = _EXPORT_CMDS[fmt] + [src, out]          # argv array, shell=False
    subprocess.run(cmd, check=True, timeout=15, capture_output=True)   # CmdInj fix
(CWE-78)
    return send_file(out, as_attachment=True, download_name=f"{f.original_name}.
{fmt}")

@files_bp.route("/upload", methods=["POST"])
@login_required
@limiter.limit("30 per minute")
def upload():
    file = request.files.get("file")
    original = secure_filename(file.filename)     # path-traversal safe
    if not original or not _allowed(original): abort(400)
    ext = original.rsplit(".", 1)[-1].lower()
    stored = f"{uuid.uuid4().hex}.{ext}"          # server-generated name
    file.save(os.path.join(current_app.config["UPLOAD_DIR"], stored))
    db.session.add(File(stored_name=stored, original_name=original,
                        owner_id=current_user.id))
    db.session.commit()
```

D.4 Native helper — VULNERABLE vs SECURE (`native/`)

Vulnerable (`thumbgen_vuln.c`, lab-only):

```
int make_preview(const char *in, size_t in_len, char *out, size_t out_cap){
    char buf[THUMB_OUT_LEN];       /* 64 bytes */
    memcpy(buf, in, in_len);       /* VULN: trusts caller length (CWE-120) */
    buf[THUMB_OUT_LEN-1]='\0';
    strcpy(out, buf);              /* VULN: ignores out_cap */
    return 0;
}
```

Secure (`thumbgen_secure.c`, deployed):

```
int make_preview(const char *in, size_t in_len, char *out, size_t out_cap){
    char buf[THUMB_OUT_LEN]; size_t i, n = in_len;
    if (in==NULL || out==NULL || out_cap==0) return 1;
    if (n > THUMB_OUT_LEN-1) n = THUMB_OUT_LEN-1;   /* FIX: clamp length */
    memcpy(buf, in, n); buf[n]='\0';                /* in-bounds NUL     */
    for (i=0;i<n;i++){ unsigned char c=(unsigned char)buf[i];
                       if(!isprint(c)) buf[i]='.'; }
    snprintf(out, out_cap, "%s", buf);              /* FIX: bounded write */
    return 0;
}
```

Python wrapper caps input to 256 bytes and never passes a client length (`app/native.py`). Build flags: `-fstack-protector-strong -D_FORTIFY_SOURCE=2`.

D.5 Authentication — `app/auth.py` (excerpt)

```
@auth_bp.route("/login", methods=["GET", "POST"])
@limiter.limit("10 per minute", methods=["POST"])      # brute-force / DoS
def login():
    if request.method == "POST":
        user =
User.query.filter_by(username=request.form.get("username","").strip()).first()
        if user is None or not user.check_password(request.form.get("password","")):
            flash("Invalid username or password.", "error")   # no enumeration
            return render_template("login.html"), 401
        login_user(user)
        return redirect(url_for("files.dashboard"))
    return render_template("login.html")
```

D.6 Security test suite — `tests/` (representative)

```
# test_idor.py
def test_idor_other_user_is_forbidden(client):
    login(client, "mallory", "mallory-pass-9")
    assert client.get("/file/1/download").status_code == 403

# test_csrf.py
def test_delete_without_csrf_token_is_rejected(client):
    login(client, "alice", "alice-password-1")
    assert client.post("/file/1/delete", data={}).status_code == 400

# test_clickjacking.py
def test_security_headers_on_public_page(client):
    r = client.get("/login")
    assert r.headers.get("X-Frame-Options") == "DENY"
    assert "frame-ancestors 'none'" in r.headers.get("Content-Security-Policy","")

# test_command_injection.py
def test_export_rejects_non_allowlisted_format(client):
    login(client, "alice", "alice-password-1")
```

```
    token = _extract_csrf(client.get("/dashboard").data)
    assert client.post("/file/1/export",
                       data={"csrf_token": token, "format": "zip; id"}).status_code
== 400

# test_auth_and_overflow.py
def test_make_preview_is_bounds_safe():
    out = make_preview(b"A" * 4096)
    assert len(out) <= 64
```

D.7 CI/CD pipeline — `.github/workflows/ci-cd.yml` (structure)

```
on: { push: {branches:[main,develop]}, pull_request: {branches:[main,develop]} }
jobs:
  build-and-test:   # deps -> make secure -> pytest -> ASAN before/after evidence
  sast-bandit:      # bandit -r app -ll   (GATE: fail on high/medium)
  sast-semgrep:     # p/security-audit + p/owasp-top-ten --error (GATE) -> SARIF
  codeql:           # matrix [python, cpp] -> SARIF to Security tab
  dast-zap:         # OWASP ZAP baseline against the running app (report)
```

The Bandit and Semgrep stages exit non-zero on high/critical findings in production code, failing the required check and (with branch protection) blocking the merge — the fail-on-critical guarantee.